

question 1 code + results:

```
using namespace std;
#include <iostream>

void ECSumArray(const int *listNums, int size)
{
    // You need to loop through the array using pointer.
    int sum = 0;
    const int *p = listNums; // pointer

    for (int i = 0; i < size; i++)
    {
        sum += *(p + i);
    }
    cout << "sum: " << sum << endl;
}

int main()
{
    int array[] = {1,2,3,5};
    ECSumArray(array, 4);
    return 0;
}
```

```
karin@Karin's_Laptop CLANG64 /c/Users/karin/Music/CSE-3150/lab3
$ clang++ ECConstPointer.cpp
ECConstPointer.cpp:1:17: warning: using directive refers to implicitly-defined namespace 'std'
    1 | using namespace std;
      | ^
1 warning generated.

karin@Karin's_Laptop CLANG64 /c/Users/karin/Music/CSE-3150/lab3
$ ./a.exe
sum: 11
```

question 2: (sorry if it's small i tried to smush the results + code onto this page)

```
#include <iostream>
using namespace std;

void ECSortFP(int *listNumbers, int size, bool (*cmp)(int, int)) {
    // your code here
    for (int i = 0; i < size - 1; i++) {
        for (int j = i + 1; j < size; j++) {
            if (!cmp(listNumbers[i], listNumbers[j])) {
                int temp = listNumbers[i];
                listNumbers[i] = listNumbers[j];
                listNumbers[j] = temp;
            }
        }
    }
}

int main() {
    int arr[] = {1,3,2,4};
    ECSortFP(arr, 4, [](int a, int b) { return a < b; });
    for(int i=0; i<4; ++i) {
        cout << arr[i] << " ";
    }
    cout << endl;

    ECSortFP(arr, 4, [](int a, int b) { return a > b; });
    for(int i=0; i<4; ++i) {
        cout << arr[i] << " ";
    }
    cout << endl;
}
```

```
karin@Karin's_Laptop CLANG64 /c/Users/karin/Music/CSE-3150/lab3
$ clang++ ECSortFuncPtr.cpp

karin@Karin's_Laptop CLANG64 /c/Users/karin/Music/CSE-3150/lab3
$ ./a.exe
1 2 3 4
4 3 2 1
```

question 3 code + results:

```
#include <iostream>
using namespace std;

// function takes a const int pointer and then modify it
void foo(const int *px)
{
    // cast to non-const and then modify the int pointed by it (you need to add a line)
    int *pcx = const_cast<int*>(px);
    *pcx = 1;
}

// do the following casting work?
int main()
{
    // cast an int to char
    char ch1 = 'a'; // no cast
    int x1 = static_cast<int>(ch1);
    cout << "x1 = " << x1 << ", ch1 = " << ch1 << endl;

    // cast a pointer to an int to a pointer to a double
    int y1 = 10;
    double *py1 = reinterpret_cast<double*>(&y1);
    cout << "*py1: " << *py1 << endl;

    // cast a pointer to char to a pointer to int
    char ch = 'a';
    int *pch = reinterpret_cast<int*>(&ch);
    cout << "*pch: " << *pch << endl;

    // cast a void pointer to point to int
    int x4 = 20;
    void *px4 = &x4;
    int *px4i = static_cast<int*>(px4);
    cout << "*px4i: " << *px4i << endl;

    // const_cast
    // cast a const int pointer to a non-const pointer to int and then modify it
    const int x5 = 0;
    int *px5 = const_cast<int*>(&x5);
    *px5 = 1;
    cout << "x5: " << x5 << ", *px5: " << *px5 << endl;

    // const_cast and function call
    const int x6 = 0;
    foo(&x6);
    cout << "x6: " << x6 << endl;

    return 0;
}
```

```
karin@Karin's_Laptop CLANG64 /c/Users/karin/Music/CSE-3150/lab3
$ clang++ ECCastTest.cpp

karin@Karin's_Laptop CLANG64 /c/Users/karin/Music/CSE-3150/lab3
$ ./a.exe
x1 = 97, ch1 = a
*py1: 2.05834e-312
*pch: -654679967
*px4i: 20
x5: 0, *px5: 1
x6: 0
```

question 4 code + results:

```
#include <string>
#include <iostream>
using namespace std;

//function that modifies a string `x` so that after the function call, `x` contains only
void TrimString(std::string &x)
{
    string res;
    for(unsigned int i=0; i<5 && i<x.size(); ++i)
    {
        res += x[i];
    }
    x = res;
}

int main()
{
    string s = "12345678Hello";
    TrimString(s);
    cout << s << endl; // prints "Hello"
    return 0;
}
```

```
karin@Karinas_Laptop CLANG64 /c/Users/karin/Music/CSE-3150/lab3
$ clang++ TrimString.cpp

karin@Karinas_Laptop CLANG64 /c/Users/karin/Music/CSE-3150/lab3
$ ./a.exe
12345
```

question 5 code + results:

```
// Reverse an integer (stored in decimal format in a string)
string ECReverseInt(const string &strNumber)
{
    string res;

    // your code goes here

    for (int i = strNumber.size() - 1; i >= 0; --i) {
        res += strNumber[i];
    }

    size_t firstNonZero = res.find_first_not_of('0');
    res = res.substr(firstNonZero);

    return res;
}

int main()
{
    cout << ECReverseInt("1234") << endl;
    cout << ECReverseInt("4321") << endl;
    cout << ECReverseInt("3210") << endl;
    cout << ECReverseInt("1000") << endl;
    cout << ECReverseInt("1002") << endl;
}
```

```
karin@Karinas_Laptop CLANG64 /c/Users/karin/
$ clang++ ECReverseInt.cpp

karin@Karinas_Laptop CLANG64 /c/Users/karin/
$ ./a.exe
4321
1234
123
1
2001
```